

Sanal Bellek: Talep Üzerine Sayfalama ve Sayfa Değiştirme

İşletim Sistemleri — Konu 7

1. Sanal Belleğin Amacı

Sanal bellek (virtual memory), bir sürecin kullandığı adres uzayını fiziksel bellekten (RAM) ayıran soyutlamadır. Süreç 0'dan başlayan kesintisiz bir adres uzayı görür; bu adreslerin fiziksel bellekte nerede durduğunu — ya da hiç durmadığını — bilmez. Eşleme donanımındaki MMU (Memory Management Unit) ile çalışma zamanında yapılır.

Bu ayrım üç şeyi mümkün kılar:

- Fiziksel bellekten büyük programlar. Programın tamamı aynı anda RAM'de olmak zorunda değildir; yalnız o an kullanılan sayfalar bellekte tutulur.
- İzolasyon. İki sürecin aynı sanal adresi farklı fiziksel çerçevelere düşer; biri diğerinin belleğini okuyamaz.
- Paylaşım. Aynı kütüphanenin kod sayfaları tek bir fiziksel kopyayla birden çok sürecin adres uzayına eşlenebilir.

1.1. Sayfa ve Çerçeve

Sanal adres uzayı sabit boyutlu sayfalara (page), fiziksel bellek aynı boyutta çerçevelere (frame) bölünür. Tipik sayfa boyutu 4 KB'tır. Sanal adres iki parçaya ayrılır: üst bitler sayfa numarası, alt bitler sayfa içi offset. Offset çevrimde değişmez; yalnız sayfa numarası bir çerçeve numarasına çevrilir.

32 bitlik bir adres uzayında 4 KB sayfa için offset 12 bit, sayfa numarası 20 bittir. Bu, süreç başına $2^{20} = 1.048.576$ sayfa girdisi demektir; her girdi 4 bayt olsa tek seviyeli bir sayfa tablosu süreç başına 4 MB yer kaplardı. Bu yüzden gerçek sistemler çok seviyeli sayfa tablosu kullanır: üst seviyedeki girdilerin çoğu boştur ve boş dalın alt tabloları hiç oluşturulmaz.

2. Talep Üzerine Sayfalama

Talep üzerine sayfalama (demand paging), bir sayfayı ancak gerçekten erişildiğinde belleğe getirme stratejisidir. Süreç başlatıldığında hiçbir sayfası bellekte olmayabilir; çalışmaya başlayınca eriştiği sayfalar teker teker yüklenir.

Sayfa tablosu girdisinde bir geçerlilik biti (valid bit) vardır. Bit 0 ise sayfa fiziksel bellekte değildir ve erişim donanım tarafından yakalanır.

2.1. Sayfa Hatası Nasıl İşlenir

Bir sayfa hatası (page fault) oluştuğunda sıra şudur:

1. MMU geçersiz girdiyi görür ve işlemciye bir tuzak (trap) üretir; kontrol çekirdeğe geçer.
2. Çekirdek erişimin meşru olup olmadığına bakar. Adres sürecin haritasında hiç yoksa bu bir programlama hatasıdır ve süreç SIGSEGV ile sonlandırılır.
3. Erişim meşruysa boş bir çerçeve bulunur. Boş çerçeve yoksa bir kurban sayfa (victim) seçilir ve gerekiyorsa diske yazılır.
4. İstenen sayfa diskten çerçeveye okunur. Bu bir G/Ç işlemidir; süreç bu sırada blocked durumuna geçer ve CPU başka bir sürece verilir.
5. Sayfa tablosu güncellenir, geçerlilik biti 1 yapılır.
6. Hataya sebep olan komut baştan çalıştırılır. Komut yarıda kaldığı yerden değil, en baştan tekrarlanır; bu yüzden komutların yeniden başlatılabilir olması donanım tasarımının bir gereğidir.

2.2. Sayfa Hatasının Maliyeti

Bellek erişimi ~100 nanosaniye, disk erişimi ~8 milisaniye mertebesindedir; arada yaklaşık 80.000 kat fark vardır. Etkin erişim süresi şu şekilde hesaplanır:

$$EAT = (1 - p) \times \text{bellek_erisimi} + p \times \text{sayfa_hatasi_maliyeti}$$

Burada p sayfa hatası olasılığıdır. p = 0.001 gibi görünürde küçük bir oran bile etkin erişim süresini yaklaşık 80 kat yavaşlatır. Bu yüzden sayfa hatası oranı yüzde mertebesinde değil, binde birin altında tutulmak zorundadır.

3. Sayfa Değiştirme Algoritmaları

Boş çerçeve kalmadığında hangi sayfanın çıkarılacağına sayfa değiştirme algoritması karar verir. Algoritmalar aynı referans dizisi (reference string) üzerinde karşılaştırılır; ölçüt üretilen sayfa hatası sayısıdır.

3.1. FIFO

Belleğe en önce giren sayfa ilk çıkar. Uygulaması en ucuz olandır: tek bir kuyruk yeter, erişim sırasında hiçbir muhasebe tutulmaz.

Zayıflığı, sayfanın ne kadar kullanıldığını hiç dikkate almamasıdır: sık kullanılan bir sayfa yalnız eski olduğu için çıkarılabilir.

FIFO ayrıca Belady anomalisi gösterir: çerçeve sayısı artırıldığında sayfa hatası sayısı artabilir. Bu sezgiye aykırıdır ve FIFO'nun bir yığın algoritması (stack algorithm) olmamasından kaynaklanır.

3.2. Optimal (OPT)

Gelecekte en uzun süre kullanılmayacak sayfayı çıkarır. Üretilbilir en düşük sayfa hatası sayısını verir, ama geleceği bilmeyi gerektirdiği için uygulanamaz. Değeri bir ölçüt olmasıdır: diğer algoritmalar "optimalden ne kadar uzak" diye ölçülür.

3.3. LRU (En Son Kullanılandan En Uzak)

En uzun süredir kullanılmayan sayfayı çıkarır. Geçmişin geleceğe benzeyeceği varsayımına dayanır ve pratikte optimale yakın sonuç verir. Belady anomalisi göstermez.

Sorunu maliyetidir: her bellek erişiminde bir zaman damgası güncellemek ya da bir bağlı listeyi yeniden düzenlemek gerekir. Bu, donanım desteği olmadan her erişimi yavaşlatır.

3.4. Clock (İkinci Şans)

LRU'nun ucuz yaklaşığıdır. Çerçeveler dairesel bir listede tutulur ve her sayfanın bir referans biti vardır; donanım sayfaya erişildiğinde biti 1 yapar.

Bir kurban aranırken işaretçi ilerler:

- Referans biti 1 ise bit 0 yapılır ve sayfaya "ikinci şans" verilir, işaretçi ilerler.
- Referans biti 0 olan ilk sayfa kurban seçilir.

Böylece son turda hiç kullanılmamış bir sayfa çıkarılır ve maliyet erişim başına tek bir bit yazmaya iner.

4. Çerçeve Tahsisi ve Thrashing

4.1. Kaç Çerçeve Verilmeli

Çerçeveler süreçler arasında eşit (equal) ya da adres uzayı boyutuyla orantılı (proportional) dağıtılabılır. Orantılı dağıtım genelde daha iyidir, ama tek başına yetmez: bir sürecin ihtiyacı zaman içinde değişir.

Değiştirme kapsamı da bir karardır:

- Yerel değiştirme (local replacement): süreç yalnız kendi çerçevelerinden birini kurban seçebilir. Süreçler birbirinin performansını bozamaz, ama bir süreç kendi darlığından kurtulamaz.
- Küresel değiştirme (global replacement): kurban tüm bellekten seçilir. Daha iyi ortalama verim sağlar, ama bir sürecin davranışı diğerinin sayfa hatası oranını belirler ve süre garantisi verilemez hale gelir.

4.2. Thrashing

Thrashing, sistemin işlem yapmak yerine zamanının çoğunu sayfa taşımakla geçirmesi durumudur. Oluşum zinciri şöyledir: bir sürece ihtiyacından az çerçeve verilir, sık sayfa hatası üretir, hata sırasında blocked olduğu için CPU kullanımı düşer.

İşletim sistemi CPU kullanımının düştüğünü görüp çok programlılık derecesini artırır — yani yeni süreçler başlatır — çerçeve başına düşen bellek daha da azalır ve durum kötüleşir. Bu geri besleme döngüsü thrashing'i bir uçurum haline getirir: yük belli bir noktayı geçtiğinde verim kademeli değil, ani olarak çöker.

Doğru tepki terstir: çok programlılık derecesi düşürülür, bazı süreçler tümüyle diske alınır (swap out).

4.3. Çalışma Kümesi Modeli

Çalışma kümesi (working set), bir sürecin son Δ bellek referansında dokunduğu farklı sayfaların kümesidir. $WSS(i)$ bu kümenin boyutudur.

Model şunu söyler: bir süreç çalışma kümesi bellekte tutulduğu sürece az sayfa hatası üretir. Toplam talep $D = \sum WSS(i)$ mevcut çerçeve sayısını aşarsa thrashing kaçınılmazdır. İşletim sistemi D'yi izleyip aşım gördüğünde bir süreci askıya alır.

Δ seçimi kritiktir: çok küçükse çalışma kümesinin tamamını kapsamaz, çok büyükse artık kullanılmayan sayfaları da içine alır.

4.4. Bellek Eşleme ve Kopyalarken Yazma

`fork()` çağrısında ebeveynin tüm adres uzayını kopyalamak pahalıdır ve genelde gereksizdir — çocuk çoğu zaman hemen `exec()` çağırır. Kopyalarken yazma (copy-on-write) bu maliyeti erteler: sayfalar başta paylaşılır ve salt okunur işaretlenir. Taraflardan biri yazmaya kalkıştığında sayfa hatası oluşur, çekirdek o sayfanın özel bir kopyasını çıkarır ve yalnız o sayfa çoğaltılır.